P1.1 Suma de cuadrados por procesos

Crea un programa que sume los cuadrados de los números del **1 al 50**, primero de forma secuencial y después repartiendo el trabajo entre procesos. Las dos versiones deben obtener **42925**. Lo importante es comprender cómo se reparte el trabajo y cómo se reúnen los resultados.

Antes de empezar

Utiliza el entorno de clase: **Python 3.14.x**, un entorno virtual y VS Code, en Windows o Linux. No necesitas instalar bibliotecas adicionales.

Puedes resolver toda la práctica en un único archivo .py, creado por ti y con el nombre que prefieras. También puedes organizarlo en varios archivos. No hay una plantilla obligatoria ni archivos auxiliares que debas descargar. Apóyate en los ejemplos trabajados en clase.

1. Calcula el resultado de forma secuencial

Genera los datos y divídelos en grupos de **7 números**, llamados lotes. Numera los lotes para poder reconocerlos. Tendrás ocho: siete completos y uno con el número 50.

Escribe una función que reciba un lote y calcule la suma de sus cuadrados. Llámala desde el programa principal para procesar todos los lotes y sumar sus resultados. Calcula a partir de los datos; no escribas 42925 directamente como respuesta.

2. Reparte el mismo trabajo entre procesos

Utiliza **ProcessPoolExecutor**, con `max_workers=2` y el contexto `spawn`. Crea un único pool para todos los lotes de cada ejecución y reutiliza la misma función de cálculo. Puedes repartirlos con `pool.map`, como en clase.

Cada resultado debe indicar **qué lote se ha calculado, el PID del trabajador, cuántos números contiene y su suma parcial**. Puedes devolver una tupla, un diccionario u otra estructura sencilla; no se exige un formato concreto.

Define la función trabajadora fuera de otras funciones y gestiona el pool con `with`. Ejecuta el archivo desde la terminal de VS Code, con el arranque protegido. Por ejemplo, si lo has agrupado en `main`:

```
if __name__ == "__main__":
    main()
```

3. Reúne y comprueba los resultados

Muestra el PID del proceso principal y los resultados de los lotes. Suma las parciales en el principal y compara en el programa el total con el de la versión secuencial.

Comprueba, mediante el código o revisando la salida, que aparece cada lote una sola vez y que se han procesado **50 números**.

Observa los PIDs: **ocho lotes no significan ocho trabajadores**. Anota qué procesos han realizado el trabajo y si un mismo PID aparece en varios lotes.

4. Haz tres pruebas adicionales

Basta con cambiar los datos y el tamaño del lote en tu programa. No necesitas un menú ni un archivo de pruebas automáticas.

Prueba	Datos y tamaño	Qué comprobar
Lote incompleto	[1, 2, 3] Tamaño 2	Dos lotes; el último contiene un número. Total 14 en ambas versiones.
Lista vacía	[] Tamaño 7	Total 0 en ambas versiones, sin errores.
Error en una tarea	[1, "error", 3] Tamaño 2	Ejecuta solo la versión con procesos. El principal recoge el error del trabajador con try/except y muestra un mensaje claro.

En la prueba de error, no presentes una suma incompleta como resultado correcto.

5. Compara los tiempos

Utiliza `perf_counter` para medir las dos versiones con los mismos datos y lotes. En la versión con procesos, incluye la creación y el cierre del pool. Deja los mensajes de pantalla fuera de la medición.

Ejecuta el caso del 1 al 50 **tres veces** y recoge los tiempos, en segundos, en una tabla. Comenta qué observas. No se exige que la versión con procesos sea más rápida. No añadas pausas con `sleep` para alterar la comparación.

6. Entrega y explica tu trabajo

Entrega el **código fuente y una memoria breve en PDF**, orientativamente de una o dos páginas. Puedes reunirlo todo en un ZIP. No incluyas el entorno virtual.

En la memoria indica tu nombre, el sistema operativo, la versión de Python y cómo ejecutar el programa. Incluye las salidas o capturas necesarias de las pruebas, la tabla de tiempos y respuestas breves a estas preguntas:

a) ¿Cuántos datos, lotes y trabajadores intervienen? ¿Qué has observado en los PIDs?

b) ¿Por qué devuelve cada trabajador su resultado en lugar de acumularlo en una variable global del principal?

c) ¿Para qué sirven `spawn`, la protección `__main__` y el bloque `with`?

d) ¿Cómo recoges los resultados? Si usas `map`, ¿su orden de entrega demuestra el orden en que terminaron las tareas?

Puedes consultar documentación y utilizar IA como apoyo; indica brevemente la ayuda utilizada. Debes poder explicar tu código. El profesor podrá pedirte una explicación o una pequeña modificación.

Calificación

Comprensión: 25 puntos; reparto y cálculo: 25; procesos y error: 20; pruebas y entrega: 20; tiempos: 10. Hasta 5 puntos adicionales por presentación, con nota final limitada a 100. Se admiten soluciones distintas que cumplan estos objetivos.